Lab4: Programming Symmetric & Asymmetric Crypto

Code:

```
import os
import time
import statistics
from pathlib import Path
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa, padding as
asym_padding
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding as sym_padding
from cryptography.hazmat.backends import default_backend
from cryptography.exceptions import InvalidSignature

backend = default_backend()


# Helpers
def get_random_bytes(n: int) -> bytes:
    return os.urandom(n)

def pkcs7_pad(data: bytes, block: int = 16) -> bytes:
    p = sym_padding.PKCS7(block * 8).padder()
    return p.update(data) + p.finalize()

def pkcs7_unpad(data: bytes, block: int = 16) -> bytes:
    p = sym_padding.PKCS7(block * 8).unpadder()
    return p.update(data) + p.finalize()


# Key handling

KEYS = {}

def ensure_keys():
    """Generate AES-128, AES-256 and RSA-2048 keys if they do not exist."""
    if not Path('aes128.key').exists():
        Path('aes128.key').write_bytes(get_random_bytes(16))
    if not Path('aes256.key').exists():
        Path('aes256.key').write_bytes(get_random_bytes(32))
    if not Path('rsa_priv.pem').exists():
        priv = rsa.generate_private_key(public_exponent=65537,
```

```
                                        key_size=2048,
                                        backend=backend)
        pem_priv = priv.private_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PrivateFormat.PKCS8,
            encryption_algorithm=serialization.NoEncryption())
        Path('rsa_priv.pem').write_bytes(pem_priv)

        pub = priv.public_key().public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo)
        Path('rsa_pub.pem').write_bytes(pub)

def load_keys():
    global KEYS
    KEYS['aes128'] = Path('aes128.key').read_bytes()
    KEYS['aes256'] = Path('aes256.key').read_bytes()
    KEYS['rsa_priv'] = serialization.load_pem_private_key(
        Path('rsa_priv.pem').read_bytes(), password=None, backend=backend)
    KEYS['rsa_pub']  = KEYS['rsa_priv'].public_key()


# AES

def aes_encrypt(plain_file: str, cipher_file: str, keylen: int, mode: str):
    key = KEYS[f'aes{keylen}']
    data = Path(plain_file).read_bytes()
    iv = get_random_bytes(16) if mode == 'CFB' else None

    cipher = Cipher(algorithms.AES(key),
                    modes.CFB(iv) if mode == 'CFB' else modes.ECB(),
                    backend=backend)
    encryptor = cipher.encryptor()

    start = time.perf_counter()
    ct = encryptor.update(pkcs7_pad(data)) + encryptor.finalize()
    elapsed = time.perf_counter() - start

    out = (iv or b'') + ct
    Path(cipher_file).write_bytes(out)
    print(f"AES-{keylen} {mode} encrypted -> {cipher_file}  ({elapsed:.6f}s)")

def aes_decrypt(cipher_file: str, keylen: int, mode: str):
    key = KEYS[f'aes{keylen}']
    ciphertext = Path(cipher_file).read_bytes()
```

```
    if mode == 'CFB':
        iv, ct = ciphertext[:16], ciphertext[16:]
        cipher_mode = modes.CFB(iv)
    else:
        ct = ciphertext
        cipher_mode = modes.ECB()

    cipher = Cipher(algorithms.AES(key), cipher_mode, backend=backend)
    decryptor = cipher.decryptor()

    start = time.perf_counter()
    padded = decryptor.update(ct) + decryptor.finalize()
    elapsed = time.perf_counter() - start

    plain = pkcs7_unpad(padded)
    print("\n--- Decrypted text ---")
    print(plain.decode(errors='ignore'))
    print(f"--- Decryption time: {elapsed:.6f}s ---\n")


# RSA

def rsa_encrypt(plain_file: str, cipher_file: str):
    pub = KEYS['rsa_pub']
    data = Path(plain_file).read_bytes()

    max_len = (pub.key_size // 8) - 2 * hashes.SHA256().digest_size - 2
    if len(data) > max_len:
        raise ValueError(f"Message too long for 2048-bit RSA (max {max_len}
bytes)")

    start = time.perf_counter()
    ct = pub.encrypt(
        data,
        asym_padding.OAEP(
            mgf=asym_padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None))
    elapsed = time.perf_counter() - start

    Path(cipher_file).write_bytes(ct)
    print(f"RSA encrypted -> {cipher_file}  ({elapsed:.6f}s)")

def rsa_decrypt(cipher_file: str):
```

```
    priv = KEYS['rsa_priv']
    ct = Path(cipher_file).read_bytes()

    start = time.perf_counter()
    pt = priv.decrypt(
        ct,
        asym_padding.OAEP(
            mgf=asym_padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None))
    elapsed = time.perf_counter() - start

    print("\n--- RSA decrypted text ---")
    print(pt.decode(errors='ignore'))
    print(f"--- Decryption time: {elapsed:.6f}s ---\n")


# RSA Signature

def rsa_sign(message_file: str, sig_file: str):
    priv = KEYS['rsa_priv']
    msg = Path(message_file).read_bytes()

    start = time.perf_counter()
    sig = priv.sign(
        msg,
        asym_padding.PSS(
            mgf=asym_padding.MGF1(hashes.SHA256()),
            salt_length=asym_padding.PSS.MAX_LENGTH),
        hashes.SHA256())
    elapsed = time.perf_counter() - start

    Path(sig_file).write_bytes(sig)
    print(f"Signature written -> {sig_file}  ({elapsed:.6f}s)")

def rsa_verify(message_file: str, sig_file: str):
    pub = KEYS['rsa_pub']
    msg = Path(message_file).read_bytes()
    sig = Path(sig_file).read_bytes()

    start = time.perf_counter()
    try:
        pub.verify(
            sig,
            msg,
```

```
            asym_padding.PSS(
                mgf=asym_padding.MGF1(hashes.SHA256()),
                salt_length=asym_padding.PSS.MAX_LENGTH),
            hashes.SHA256())
        print("Signature is VALID")
    except InvalidSignature:
        print("Signature is INVALID")
    elapsed = time.perf_counter() - start
    print(f"Verification time: {elapsed:.6f}s")


# SHA-256

def sha256_hash(file_path: str):
    digest = hashes.Hash(hashes.SHA256(), backend=backend)
    with open(file_path, 'rb') as f:
        while chunk := f.read(8192):
            digest.update(chunk)
    print(f"SHA-256: {digest.finalize().hex()}")


# Benchmark

def benchmark():
    print("\n=== BENCHMARK STARTED ===")
    # ---- AES (1 MiB, 5 runs) ----
    test_data = get_random_bytes(1_048_576)          # 1 MiB
    Path('bench_plain.bin').write_bytes(test_data)

    aes_results = {}
    for keylen in (128, 256):
        for mode in ('ECB', 'CFB'):
            times = []
            for _ in range(5):
                out = f'bench_aes_{keylen}_{mode}.bin'
                start = time.perf_counter()
                aes_encrypt('bench_plain.bin', out, keylen, mode)
                times.append(time.perf_counter() - start)
                Path(out).unlink(missing_ok=True)
            mean = statistics.mean(times)
            aes_results[(keylen, mode)] = mean
            print(f"AES-{keylen} {mode}: {mean:.6f}s (mean of 5)")

    # ---- RSA (200-byte msg, 5 key sizes, 5 runs) ----
    msg = get_random_bytes(200)
```

```
    Path('bench_msg.bin').write_bytes(msg)

    rsa_sizes = [1024, 2048, 3072, 4096, 6144]
    rsa_enc = []
    rsa_dec = []

    for size in rsa_sizes:
        priv = rsa.generate_private_key(65537, size, backend)
        pub  = priv.public_key()
        enc_t, dec_t = [], []
        for _ in range(5):
            # encrypt
            start = time.perf_counter()
            ct = pub.encrypt(
                msg,
                asym_padding.OAEP(mgf=asym_padding.MGF1(hashes.SHA256()),
                                  algorithm=hashes.SHA256(),
                                  label=None))
            enc_t.append(time.perf_counter() - start)

            # decrypt
            start = time.perf_counter()
            priv.decrypt(
                ct,
                asym_padding.OAEP(mgf=asym_padding.MGF1(hashes.SHA256()),
                                  algorithm=hashes.SHA256(),
                                  label=None))
            dec_t.append(time.perf_counter() - start)

        e_mean = statistics.mean(enc_t)
        d_mean = statistics.mean(dec_t)
        rsa_enc.append(e_mean)
        rsa_dec.append(d_mean)
        print(f"RSA-{size} enc:{e_mean:.6f}s  dec:{d_mean:.6f}s")

    # clean up
    for f in ['bench_plain.bin', 'bench_msg.bin']:
        Path(f).unlink(missing_ok=True)

    print("=== BENCHMARK FINISHED ===\n")

# Menu

def menu():
    ensure_keys()
```

```
    load_keys()

    while True:
        print("\n=== Crypto Lab ===")
        print("1) AES encrypt   2) AES decrypt")
        print("3) RSA encrypt   4) RSA decrypt")
        print("5) RSA sign      6) RSA verify")
        print("7) SHA-256 hash  8) Benchmark")
        print("9) Quit")
        choice = input("> ").strip()

        if choice == '9':
            break

        # ---------- AES encrypt ----------
        elif choice == '1':
            keylen = input("Key length (128/256): ").strip()
            mode   = input("Mode (ECB/CFB): ").strip().upper()
            plain  = input("Plaintext file: ").strip()
            cipher = input("Ciphertext output file: ").strip()
            aes_encrypt(plain, cipher, int(keylen), mode)

        # ---------- AES decrypt ----------
        elif choice == '2':
            keylen = input("Key length (128/256): ").strip()
            mode   = input("Mode (ECB/CFB): ").strip().upper()
            cipher = input("Ciphertext file: ").strip()
            aes_decrypt(cipher, int(keylen), mode)

        # ---------- RSA encrypt ----------
        elif choice == '3':
            plain  = input("Plaintext file: ").strip()
            cipher = input("Ciphertext output file: ").strip()
            rsa_encrypt(plain, cipher)

        # ---------- RSA decrypt ----------
        elif choice == '4':
            cipher = input("Ciphertext file: ").strip()
            rsa_decrypt(cipher)

        # ---------- RSA sign ----------
        elif choice == '5':
            msg = input("Message file: ").strip()
            sig = input("Signature output file: ").strip()
            rsa_sign(msg, sig)
```

```
        # ---------- RSA verify ----------
        elif choice == '6':
            msg = input("Message file: ").strip()
            sig = input("Signature file: ").strip()
            rsa_verify(msg, sig)

        # ---------- SHA-256 ----------
        elif choice == '7':
            f = input("File to hash: ").strip()
            sha256_hash(f)

        # ---------- Benchmark ----------
        elif choice == '8':
            benchmark()

        else:
            print("Invalid option - try again.")

# ------------------------------------------------------------------
if __name__ == '__main__':
    menu()
```

Outcome:

=== Crypto Lab ===

1) AES encrypt 2) AES decrypt

3) RSA encrypt 4) RSA decrypt

5) RSA sign 6) RSA verify

7) SHA-256 hash 8) Benchmark

9) Quit

> 1

Key length (128/256): 128

Mode (ECB/CFB): CFB

Plaintext file: plain.txt

Ciphertext output file: enc.bin

AES-128 CFB encrypted -> enc.bin (0.000981s)

=== Crypto Lab ===

1) AES encrypt   2) AES decrypt

3) RSA encrypt   4) RSA decrypt

5) RSA sign   6) RSA verify

7) SHA-256 hash 8) Benchmark

9) Quit

> 2

Key length (128/256): 128

Mode (ECB/CFB): CFB

Ciphertext file: enc.bin

--- Decrypted text ---

Hello, this is a secret message!

--- Decryption time: 0.000016s ---

=== Crypto Lab ===

1) AES encrypt   2) AES decrypt

3) RSA encrypt   4) RSA decrypt

5) RSA sign 6) RSA verify

7) SHA-256 hash 8) Benchmark

9) Quit

> 3

Plaintext file: plain.txt

Ciphertext output file: rsa.bin

RSA encrypted -> rsa.bin (0.039342s)

=== Crypto Lab ===

1) AES encrypt 2) AES decrypt

3) RSA encrypt 4) RSA decrypt

5) RSA sign 6) RSA verify

7) SHA-256 hash 8) Benchmark

9) Quit

> 4

Ciphertext file: rsa.bin

--- RSA decrypted text ---

Hello, this is a secret message!

--- Decryption time: 0.010793s ---

=== Crypto Lab ===

1) AES encrypt 2) AES decrypt

3) RSA encrypt 4) RSA decrypt

5) RSA sign 6) RSA verify

7) SHA-256 hash 8) Benchmark

9) Quit

> 5

Message file: plain.txt

Signature output file: sig.bin

Signature written -> sig.bin (0.019072s)

=== Crypto Lab ===

1) AES encrypt 2) AES decrypt

3) RSA encrypt 4) RSA decrypt

5) RSA sign 6) RSA verify

7) SHA-256 hash 8) Benchmark

9) Quit